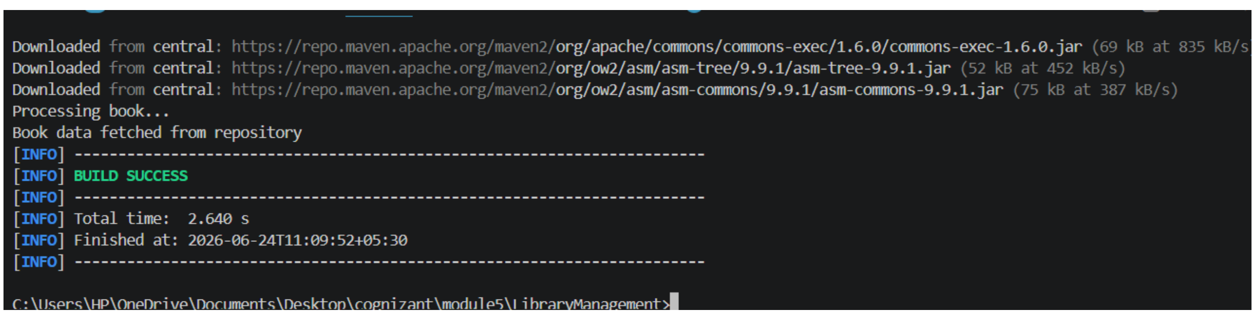
Exercise 1: Configuring a Basic Spring Application

Scenario:

Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

Steps:

1. **Set Up a Spring Project:**
    - Create a Maven project named **LibraryManagement**.
    - Add Spring Core dependencies in the **pom.xml** file.
2. **Configure the Application Context:**
    - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
    - Define beans for **BookService** and **BookRepository** in the XML file.
3. **Define Service and Repository Classes:**
    - Create a package **com.library.service** and add a class **BookService**.
    - Create a package **com.library.repository** and add a class **BookRepository**.
4. **Run the Application:**
    - Create a main class to load the Spring context and test the configuration.



Exercise 2: Implementing Dependency Injection

Scenario:

In the library management application, you need to manage the dependencies between the BookService and BookRepository classes using Spring's IoC and DI.

Steps:

1. **Modify the XML Configuration:**
    - Update **applicationContext.xml** to wire **BookRepository** into **BookService**.
2. **Update the BookService Class:**
    - Ensure that **BookService** class has a setter method for **BookRepository**.

3. **Test the Configuration:**
    - Run the **LibraryManagementApplication** main class to verify the dependency injection.

```
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Processing book...
Book data fetched from repository
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.739 s
[INFO] Finished at: 2026-06-24T11:13:52+05:30
[INFO] ------------------------------------------------------------------------

C:\Users\HP\OneDrive\Documents\Desktop\cognizant\module5\LibraryManagement>
```

Exercise 3: Implementing Logging with Spring AOP

Scenario:

The library management application requires logging capabilities to track method execution times.

Steps:

1. **Add Spring AOP Dependency:**
    - Update **pom.xml** to include Spring AOP dependency.
2. **Create an Aspect for Logging:**
    - Create a package **com.library.aspect** and add a class **LoggingAspect** with a method to log execution times.
3. **Enable AspectJ Support:**
    - Update **applicationContext.xml** to enable **AspectJ** support and register the aspect.
4. **Test the Aspect:**
    - Run the **LibraryManagementApplication** main class and observe the console for log messages indicating method execution times.

```
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Processing book...
Book data fetched from repository
Method: processBook executed in 11 ms
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.923 s
[INFO] Finished at: 2026-06-24T11:19:38+05:30
[INFO] ------------------------------------------------------------------------

C:\Users\HP\OneDrive\Documents\Desktop\cognizant\module5\LibraryManagement>
```

Exercise 4: Creating and Configuring a Maven Project

Scenario: You need to set up a new Maven project for the library management application and add Spring dependencies.

Steps:

1. **Create a New Maven Project:**
   - Create a new Maven project named **LibraryManagement**.
2. **Add Spring Dependencies in pom.xml:**
   - Include dependencies for Spring Context, Spring AOP, and Spring WebMVC.
3. **Configure Maven Plugins:**
   - Configure the Maven Compiler Plugin for Java version 1.8 in the pom.xml file.

```
PROBLEMS 82   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS   SPELL CHECKER 6   SQL HIS
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Processing book...
Book data fetched from repository
Method: processBook executed in 10 ms
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.882 s
[INFO] Finished at: 2026-06-24T11:26:47+05:30
[INFO] ------------------------------------------------------------------------

C:\Users\HP\OneDrive\Documents\Desktop\cognizant\module5\LibraryManagement>
```

Exercise 5: Configuring the Spring IoC Container

Scenario:

The library management application requires a central configuration for beans and dependencies.

Steps:

1. **Create Spring Configuration File:**
   - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
   - Define beans for **BookService** and **BookRepository** in the XML file.
2. **Update the BookService Class:**
   - Ensure that the **BookService** class has a setter method for **BookRepository**.
3. **Run the Application:**
   - Create a main class to load the Spring context and test the configuration.

```
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Processing book...
Book data fetched from repository
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.709 s
[INFO] Finished at: 2026-06-24T11:35:36+05:30
[INFO] ------------------------------------------------------------------------

C:\Users\HP\OneDrive\Documents\Desktop\cognizant\module5\LibraryManagement>
```

Exercise 6: Configuring Beans with Annotations

Scenario:

You need to simplify the configuration of beans in the library management application using annotations.

Steps:

1. **Enable Component Scanning:**
    - Update **applicationContext.xml** to include component scanning for the **com.library** package.
2. **Annotate Classes:**
    - Use **@Service** annotation for the **BookService** class.
    - Use **@Repository** annotation for the **BookRepository** class.
3. **Test the Configuration:**
    - Run the **LibraryManagementApplication** main class to verify the annotation-based configuration.

```
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Processing book...
Book data fetched from repository
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.790 s
[INFO] Finished at: 2026-06-24T11:39:12+05:30
[INFO] ------------------------------------------------------------------------

C:\Users\HP\OneDrive\Documents\Desktop\cognizant\module5\LibraryManagement>
```

Exercise 7: Implementing Constructor and Setter Injection

Scenario:

The library management application requires both constructor and setter injection for better control over bean initialization.

Steps:

1. **Configure Constructor Injection:**
    - Update applicationContext.**xml** to configure constructor injection for **BookService**.
2. **Configure Setter Injection:**
    - Ensure that the **BookService** class has a setter method for **BookRepository** and configure it in **applicationContext.xml**.
3. **Test the Injection:**
    - Run the **LibraryManagementApplication** main class to verify both constructor and setter injection.

```
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Constructor Injection used
Setter Injection used
Processing book...
Book data fetched from repository
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.709 s
[INFO] Finished at: 2026-06-24T11:42:10+05:30
[INFO] ------------------------------------------------------------------------

C:\Users\HP\OneDrive\Documents\Desktop\cognizant\module5\LibraryManagement>
```

Exercise 8: Implementing Basic AOP with Spring

Scenario:

The library management application requires basic AOP functionality to separate cross-cutting concerns like logging and transaction management.

Steps:

1. **Define an Aspect:**
    - Create a package **com.library.aspect** and add a class **LoggingAspect**.
2. **Create Advice Methods:**
    - Define advice methods in **LoggingAspect** for logging before and after method execution.
3. **Configure the Aspect:**
    - Update **applicationContext.xml** to register the aspect and enable **AspectJ** auto-proxying.
4. **Test the Aspect:**
    - Run the **LibraryManagementApplication** main class to verify the AOP functionality.

```
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Processing Book...
Book Repository Working
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.911 s
[INFO] Finished at: 2026-06-24T11:55:26+05:30
[INFO] ------------------------------------------------------------------------

C:\Users\HP\OneDrive\Documents\Desktop\cognizant\module5\LibraryManagement>
```

Exercise 9: Creating a Spring Boot Application

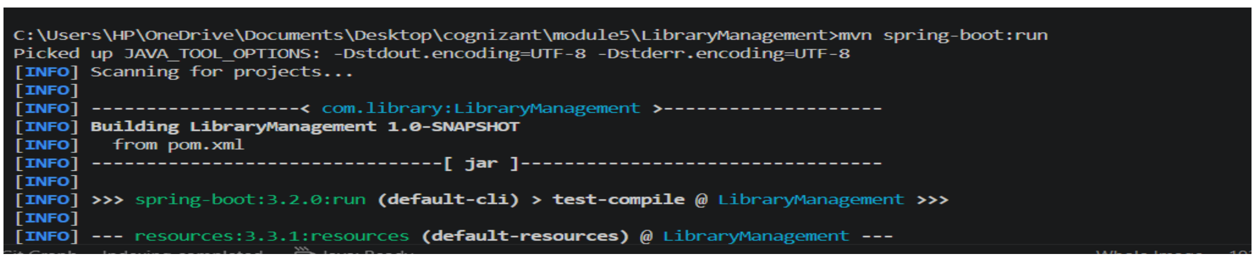
Scenario:

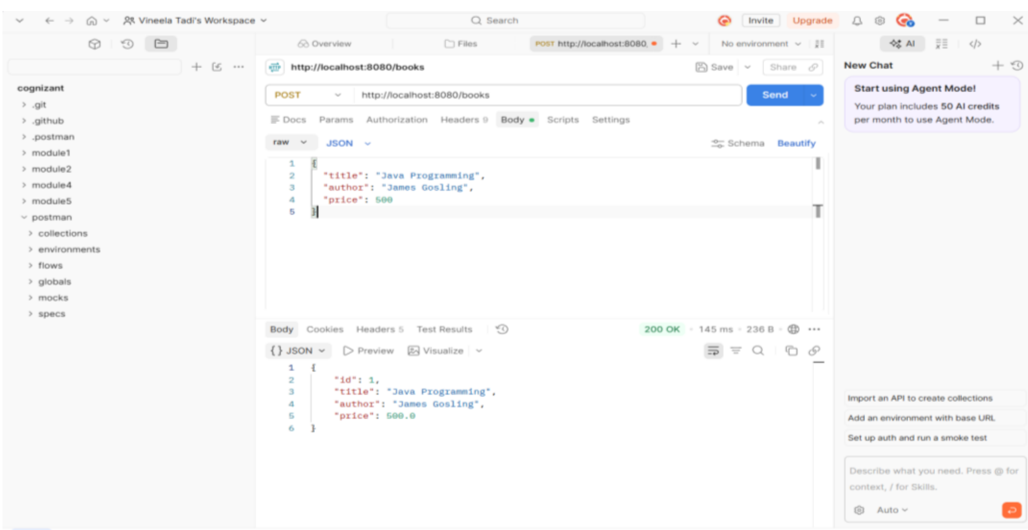
You need to create a Spring Boot application for the library management system to simplify configuration and deployment.

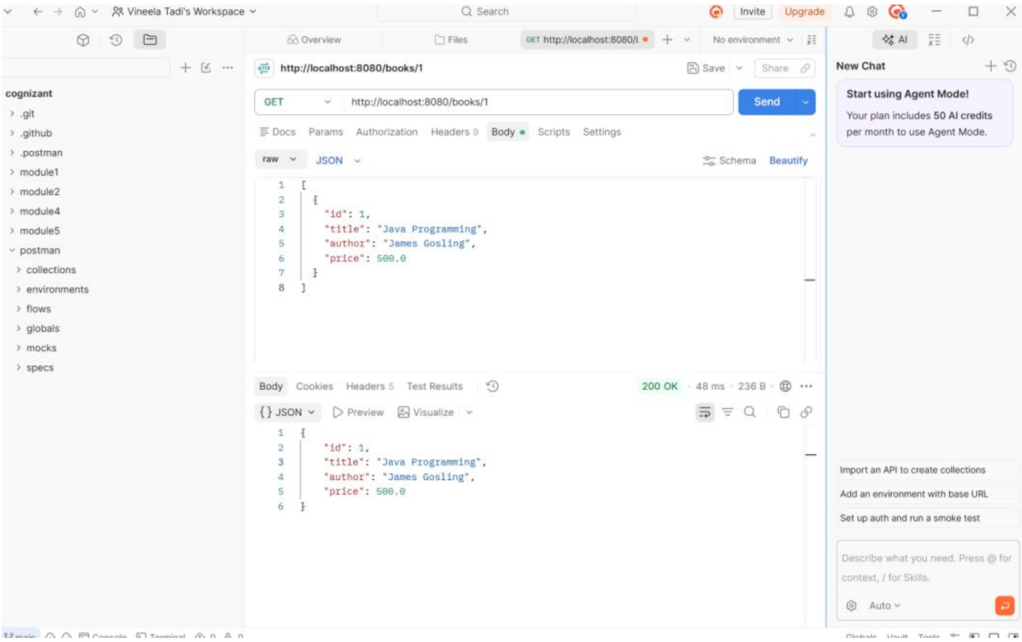
Steps:

1. **Create a Spring Boot Project:**
    - Use **Spring Initializr** to create a new Spring Boot project named **LibraryManagement**.

2. **Add Dependencies:**
    - Include dependencies for **Spring Web, Spring Data JPA, and H2 Database**.
3. **Create Application Properties:**
    - Configure database connection properties in **application.properties**.
4. **Define Entities and Repositories:**
    - Create **Book** entity and **BookRepository** interface.
5. **Create a REST Controller:**
    - Create a **BookController** class to handle CRUD operations.
6. **Run the Application:**
    - Run the Spring Boot application and test the REST endpoints.






Vineela Tadi's Workspace
Search
Invite
Upgrade
Overview
Files
GET http://localhost:8080/
No environment
AI
cognizant
.git
.github
.postman
module1
module2
module4
module5
postman
collections
environments
flows
globals
mocks
specs
http://localhost:8080/books/1
Save
Share
GET
http://localhost:8080/books/1
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
raw
JSON
Schema
Beautify
[
{
"id": 1,
"title": "Java Programming",
"author": "James Gosling",
"price": 500.0
}
]
Body
Cookies
Headers 5
Test Results
200 OK
48 ms
236 B
{ } JSON
Preview
Visualize
{
"id": 1,
"title": "Java Programming",
"author": "James Gosling",
"price": 500.0
}
New Chat
Start using Agent Mode!
Your plan includes 50 AI credits per month to use Agent Mode.
Import an API to create collections
Add an environment with base URL
Set up auth and run a smoke test
Describe what you need. Press @ for context, / for Skills.
Auto